

FSD

Semester 1, 2026

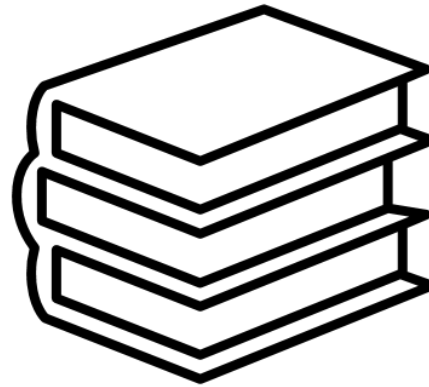
Week 03

useState, useEffect hooks Conditional rendering
Forms and
Incorporating data

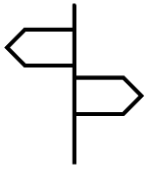
Dipto Pratyaksa dipto.pratyaksa@rmit.edu.au

Before we start

Lectorial 2 revision
Assignment 1 FAQ
Lectorial 3 Tutorial/Lab
Expectations



In week 2 we covered



- **Components- functional**
- **Interaction between components and *props***
- **Introduction to event handling**
- **useState hook**
- **Styling in React apps**

Explore deeper:

- **localStorage**
- **Login Log Out function**



Data storage

- In any web application, you will need to store data one way or the other
- Examples- store user details, product preferences, items bought, etc.
- The data storage may be temporary or persistent
- We will learn the data storage via the databases in the latter half of this course where we will use MS SQL Server database (*more for assignment 2*).
- But before that, there are other ways of storing data
 - Data structures in JS
 - HTML5's localStorage object

You will need these for assignment 1

Data storage with data structures

- Let's revisit Example 6 from last week – here we will use a data structure and localStorage to store the data.
- Week 2 example6



- Simple login-logout system
- It will also be covered in this week's lab

Assignment 1 *discussion*

- It is online
- Deadline: *Check Canvas*
- Worth 25%
- To be completed in a group of 2
- Partial React website prototype
- Based on week(s) 1-4
- DO NOT WRITE CLASS COMPONENTS- you will get a ZERO for the whole assignment.
- Discussion..

Assignment 1 *FAQ*

- **Q:** Can I team up with someone I know from another prac class?
- Yes, but you need to be consistent, and seek feedback from the same tutor that you register your group with. This tutor is your client and your assessor.

Assignment 1 *FAQ*

- **Q:** My desired prac class is full can I attend another prac?
- Yes, you can in fact attend more than one prac classes to learn from other tutors.
- Please remember: your assessment marker for A1 and A2 is the tutor that you register your group with
- Assessment marker for A3 is your default tutor. For A3 you must attend the class that you are officially enrolled in, in Week 12.

Assignment 1 *FAQ*

- **Q:** Can I use a fancy name for my team name on GitHub Classroom?
- No. Your team name on GitHub Classroom must match exactly the name you joined on Canvas

Assignment 1 *FAQ*

- **Q:** How / where can I find a team mate?
 1. Attend your prac, get acquainted with a few class mates and ask one of them if they want to team up.
 2. Use Canvas Discussion forum, share which prac class you are from, invite them to join your team.
 3. Only if you really cannot find anybody by end of this week, join someone else in another prac.

Q: What if I'm the odd one out?

You will be paired up in a random vacant group, or asked to work alone.

Assignment 1 *FAQ*

- **Q:** I'm a Masters student, can I team up with a Bachelors student?
- **No**, because you are enrolled in different course codes, with different tutors, different grade reports, different channels, and different assignment specification (PG students has additional tasks to do)

Segment 1

useState hook *again*

useEffect hook

More on components:

conditional rendering,

form handling

router

So what is a *hook*?



- **What is a Hook?**

- A *Hook* is a special function that lets you “hook into” React features. For example, *useState* is a Hook that lets you add React state to function components.

- **When would I use a Hook?**

- If you write a function component and realize you need to add some state to it, previously you had to convert it to a class. Now you can use a Hook inside the existing function component.

Reference:

<https://react.dev/reference/react>

State

- *State: helps a component to become dynamic and interactive*
- It determines how a component will render and behave
- While *props* are used to pass information down the component tree, *state* is used to alter information over time
 - Do you remember *props* are readonly!

- *Think of this scenario*
 - Whenever a user types something into an HTML input field, the user may want to see this typed information (state) displayed somewhere else in the application.
 - Therefore we need some way to change information over time and, what's more important, to notify React to re-render its component(s) again.
- *This is where state comes into the picture!!*

Do you remember useState() *hook*?

1. Call useState() hook to enable state in a functional component.
2. The first argument of the useState(initialValue) is the state's initial value.
3. [state, setState] = useState(initialValue) returns an array of 2 items: the state value and a state updater function.

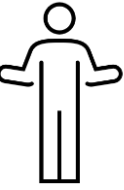
Do you remember `useState()` *hook*?

4. Invoking the state updater function `setState(newState)` with the new value updates the state.

- Alternatively, you can invoke the state updater with a callback `setState(prev => next)`, which returns the new state based on previous.

5. After the state updater is called, React makes sure to re-render the component so that the new state becomes actual.

useState *hook- when is the state updated?*



- Look at this code snippet

```
const MyComp = () => {  
  const [counter, setCounter] = useState(0);  
  onClick = () => setCounter(prev => prev + 1);  
  return <button onClick={onClick}>Click me</button>  
}
```

- Every hook has an update queue.
 - When you call the setCounter function, React doesn't call the updater function immediately, but saves it inside the *queue*, and schedules a re-render.
- The above code is simple but in reality a bigger web application may not be!

useState *hook*: when is state updated?

- There might be more updates in the code, to this hook (*shown on previous slide*), other hooks, or even hooks in other components in the tree.
- All of these updates are queued - nothing is computed immediately.
- Finally, React re-renders all components that were scheduled to be rendered, top-down. But the state updates are still not performed.
- It's only when useState actually runs, during the render function, that React runs each action in the queue, updates the final state, and returns it back.
- This is called *lazy computation* - React will calculate the new state only when it actually needs it.

useState *hook*: when is state updated?

- So when does React say: "OK, enough queueing, let me do something"?
- Whenever there's an event handler (onClick, onKeyPress, etc.) React runs the provided callback inside a *batch*.
- The batch is synchronous, it runs the callback, and then flushes all the renders that were scheduled

```
const MyComp = () => {  
  const [counter, setCounter] = useState(0);  
  
  onClick = () => { // batch starts  
    setCounter(prev => prev + 1); // schedule render  
    setCounter(prev => prev + 1); // schedule render  
  } // only here the render will run  
  return <button onClick={onClick}>Click me</button>  
}
```

Lectorial Exercise



- if React computes the new state during render time,
 - how can it bail out of render if the state didn't change?



useState is very flexible

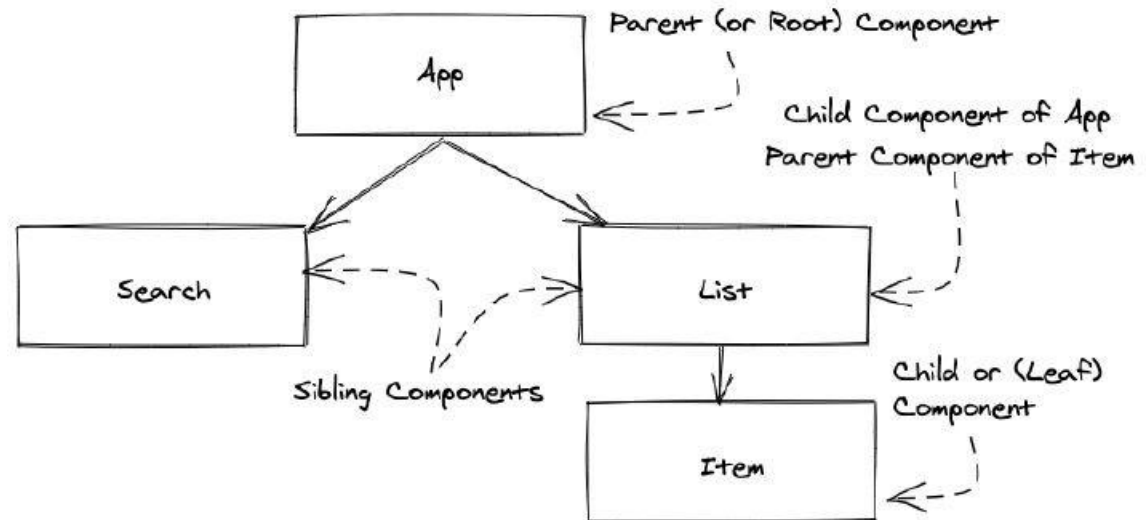
- Multiple state variables may be used and updated from within a functional component
- As opposed to strings and numbers, you could also use an object as the initial value passed to useState.
- As opposed to just passing an initial state value, state could also be initialised from a function
- Time to look at a code example
- example1





Week 3 prac activity 1 demo

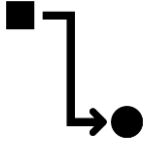
Use case for useState hook for displaying list of students



useEffect *hook*

- The Effect Hook lets you perform side effects in function components.
- *Side-effects ‘eh!!’. do I have to learn something new?*
 - *“There won’t be much to learn. In fact, we’ll spend most of our time unlearning.” - Dan Abramov*
- What are effects?
- Examples
 - Fetching data
 - Reading from local storage Registering and deregistering event listeners
 - Manually changing DOM elements ...

Effects



- Effects are always executed after render
- Hooks documentation by Facebook states-
 - Mutations, subscriptions, timers, logging, and other side effects are not allowed inside the main body of a function component (referred to as React's render phase).
 - Doing so will lead to confusing bugs and inconsistencies in the UI.
 - Instead, use *useEffect*.
 - The function passed to `useEffect` will run after the render is committed to the screen.
- To opt out or skip effects, you have to understand basic JavaScript concepts about values

useEffect *hook*

- This hook accepts a function that contains imperative, possibly effectful code.
 - `useEffect(didUpdate);`
- Often, effects create resources that need to be cleaned up before the component leaves the screen, such as a subscription or timer ID.
 - To do this, the function passed to `useEffect` may return a clean-up function

useEffect *hook*

- To understand this hook fully, one needs to explore these scenarios:
 - Running side effects after every render
 - Running an effect only when a component mounts
 - Cleaning up side-effects by returning a function
 - Controlling when an effect runs by specifying dependencies
- Time to look at some of above via examples of this hook

 example2

Segment 2

**More on components:
conditional rendering,
form handling
router**

Conditional rendering

- You can create distinct components that encapsulate behaviour you need.
 - Then, you can render only some of them, depending on the state of your application.
- Conditional rendering in React works the same way conditions work in JavaScript.
 - Use JavaScript operators like `if` or the conditional operator to create elements representing the current state, and let React update the UI to match them.
- It is a very handy concept when you create a web app

Conditional rendering- *various ways*

- if...else statement
- switch statement
- if...else with element variable
- Ternary operator
- Logical && operator

Conditional rendering

- Time to look at examples
- example3



Form handling

- *Form handling is slightly different in React*
- You need to know the concept of *controlled components*
- In HTML, form elements such as `<input>`, `<textarea>`, and `<select>` typically maintain their own state and update it based on user input.
- In React, we will handle forms with *useState* hook
- An input form element whose value is controlled by React is called a “controlled component”.
- example4 and example5



Self-exercise



- Why not style the form using react bootstrap module
- *Start here:*
 - <https://react-bootstrap.github.io/forms/overview/>



Pages Router

- Routers help create and navigate between the different URLs that make up your web application.
- They allow your user to move between the components of your app i.e. redirect to another page/component
- NextJS has a built in file system router, that is we can use files to determine the routes
- For example if we have a file in our app, located at /pages/hello.tsx, this will be accessible at /hello
- <https://nextjs.org/docs/pages/building-your-application/routing/pages-and-layouts>
- Let's have a look at basic routing, more advanced routing next week (Example 6)



Next week

- More on React components
 - Components interacting
- useEffect hook,
- custom hooks,
- Routing again and
- form handling yet again